
响应长度感知与序列调度： 由 LLM 赋能的 LLM 推理流水线

Zangwei Zheng¹, Xiaozhe Ren², Fuzhao Xue¹, Yang Luo¹, Xin Jiang², Yang You¹

¹Department of Computer Science, National University of Singapore

²Noah's Ark Lab, Huawei.

{zangwei, f-xue, yangluo, youy}@comp.nus.edu.sg; {renxiaoze, jiang.xin}@huawei.com

<https://github.com/zhengzangw/Sequence-Scheduling>

摘要

大型语言模型 (LLM) 改变了人工智能领域，在多种任务上展现出前所未有的能力。然而，LLM 推理的计算成本很高。本文提出一种利用 LLM 自身能力的高效推理流水线。我们首先挖掘 LLM 准确感知并预测响应长度的潜力，所需额外开销很小。在此基础上，提出高效的序列调度技术，把响应长度相近的查询聚合成微批次。我们在真实指令数据集上，使用基于 LLaMA 的模型评估该方法；相对于原始批量推理，推理吞吐量提高了 86%，且未损害效果。本文方法与其他推理加速技术相互独立，因此可以作为现有 LLM 推理工具的补充，例如 FlashAttention 和量化。

1 引言

大型语言模型 (LLM) [2, 6, 15, 19] 改变了自然语言处理 (NLP) 领域，在机器翻译 [23]、问答 [28]、文本摘要 [38] 等任务中取得了显著成果。然而，推理成本极高 [1, 27]，使大规模部署面临挑战。像已经部署的 ChatGPT [19] 那样处理数百万个查询，需要大量计算资源。因此，降低 LLM 推理成本已成为近年来的重要研究方向。

在真实场景中，不同查询的响应长度差异很大。如图 2a 所示，虽然不同模型的响应长度分布略有差别，但共同特点是覆盖范围很广。因此，在批量执行 LLM 推理时，把响应长度不同的序列放在一起会造成低效率：短序列被迫等待长序列完成，浪费计算。图 1 左侧展示了这一问题，冗余 token 占全部生成 token 的 66%。考虑到推理的二次时间复杂度，这种浪费给推理过程带来很大负担。

人能根据对问题的理解估计答案长度。例如，“法国的首都哪里？”通常比“你能解释法国大革命的历史吗？”得到更短的回答。我们观察到，ChatGPT、Claude 等经过指令微调的 LLM 也具备一定的响应长度感知能力。甚至 LLaMA-7B 这样较小的模型，在长度预测数据集上进行指令微调后，也能获得这一能力。尽管同一指令多次采样的输出长度会变化，见图 2b，这些模型仍能较好地感知其响应长度。

利用 LLM 的响应长度感知能力，可以改善微批次内的指令调度。这是软件与硬件协同设计的例子：LLM 自身的能力帮助加速 LLM 推理。本文序列调度系统依据感知的响应长度对查询分组，减少计算浪费。为进一步提高效率，我们引入失败收集与重新计算策略，以及可变批大小。这些技术补充了序列调度系统，继续提高推理吞吐量。

为评估本文方法的效果，我们在真实指令数据集上实验，使用经过指令微调的 LLaMA 模型 [4]，即 Vicuna（基础模型见 [34]）。本文长度预测模块在准确估计响应长度方面超过以往方法。相对于原始推理过程，序列调度系统的推理吞吐量提高了 86%，同时保持输出质量。这些结果表明，结合响应长度感知的序列调度，可以补充现有大型语言模型推理工具，例如 FlashAttention [7] 和量化 [8, 11, 35]。

本文的贡献如下：

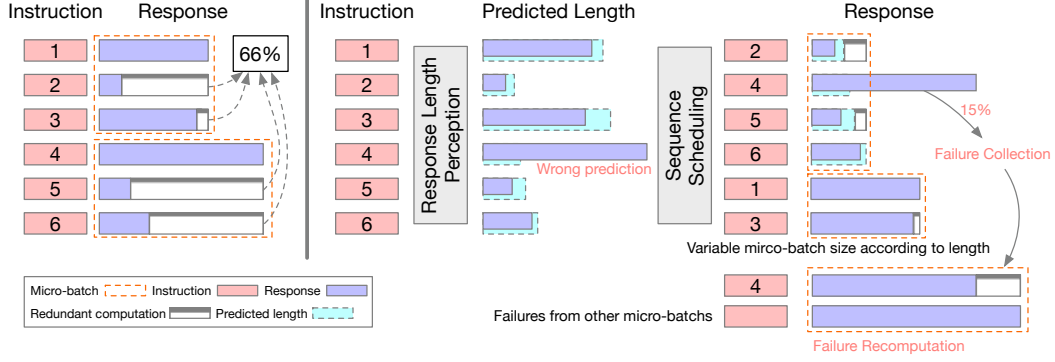
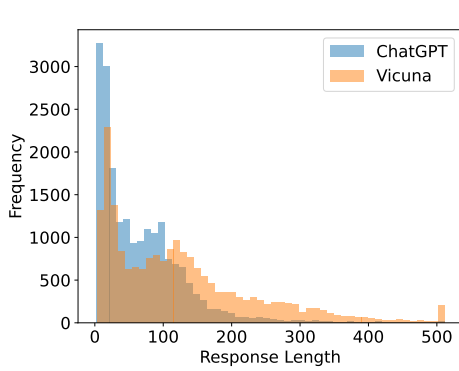
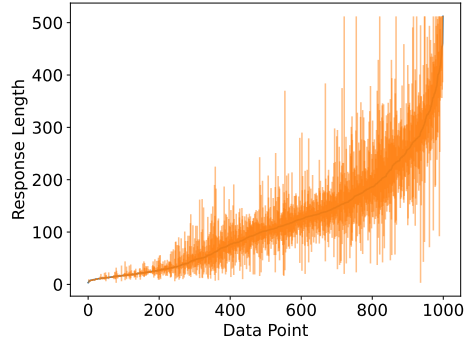


图 1: 左: 将长短响应放入同一批次, 原始批量推理产生 66% 的冗余 token, 降低性能。右: 本文序列调度流水线。响应长度感知模块先估计每条指令的响应长度; 序列调度把预测长度相近的指令分组, 并为较短响应使用更大的批大小; 失败收集与重新计算策略避免预测错误降低性能。图内英文: Instruction 为指令, Response 为响应, Response Length Perception 为响应长度感知, Predicted Length 为预测长度, Sequence Scheduling 为序列调度, Micro-batch 为微批次; Variable micro-batch size according to length 表示按长度改变微批大小, Wrong prediction 为预测错误, Failure Collection 为失败收集, Failure Recomputation 为失败重新计算, Failures from other micro-batches 为来自其他微批次的失败请求, Redundant computation 为冗余计算。



(a) ChatGPT 与 Vicuna 在 1 万条指令上的响应长度分布。长度超过 512 的响应被截断。



(b) 1000 条指令各生成 3 次时的平均长度分布; 误差条表示各次生成的最大长度与最小长度。

图 2: 响应长度及其波动的分布。图内英文: Response Length 为响应长度, Frequency 为频数, Data Point 为数据点。

- 我们研究 LLM 的响应长度感知能力, 并展示指令微调能够增强这一能力。
- 我们提出名为序列调度的新 LLM 推理流水线, 利用 LLM 的响应长度感知能力, 将长度相近的查询分组, 减少计算浪费并提高吞吐量, 同时不损害效果。
- 我们在真实指令数据集上使用 Vicuna-7B 进行了完整实验。相对于原始推理过程, 本文方法使推理吞吐量提高了 86%。

2 相关工作

大型语言模型即服务。 大型语言模型 (LLM) [2, 6, 15, 19] 通过扩大模型规模, 成功构建了强大的基础模型。借助指令微调 [25], LLM 能与人的需求对齐, 并以服务形式提供实际应用。原文发表时, ChatGPT [19]、PaLM [6] 等已作为服务部署于 Bing、Bard, 每天执行大量推理。因此, 降低 LLM 推理成本是重要研究方向。

高效 LLM 推理。 近年来, 高效 LLM 推理技术受到越来越多关注 [18]。内核融合 [5, 7] 使用高度优化的内核来减少内存访问、加快计算。流水线并行 [17, 30]、张量并行 [22, 30] 等

表 1: “预先感知”(PiA) 的生成示例。经过指令学习的模型能遵循 PiA 指令; 较小的模型 (Vicuna) 可能无法继续生成响应。

提示:

为儿童创作一道有趣的数学题。

在回答上面的指令之前, 你必须预测响应长度。请在第一行输出响应的预计单词数, 然后换行, 回答这条指令。

GPT-4:

预计响应长度: 60 个单词。

当然, 下面是一道有趣的数学题: 篮子里有 7 个苹果。一只友好的松鼠走过来, 并且 ...

ChatGPT:

4 个单词

如果你有 10 个苹果, 给朋友 3 个, 还剩多少个? <EOS>

Claude:

225

下面是一道给儿童的有趣数学题: 杂耍演员 Jenny 正在练习节目。她一只手拿着 ...

Vicuna:

101 <EOS>

方法将工作负载分布到多张 GPU, 实现 LLM 推理的高效扩展。量化 [8, 11, 35] 则通过压缩 LLM 参数提高推理效率。此外, 还有优化 LLM 批处理的研究 [3, 10, 29]。例如, [3] 关注少样本场景的查询组批, [10] 提出按输入长度将句子分组。对于 LLM, 生成阶段的成本高于提示的前向处理; 本文关注生成过程, 按预测输出长度对句子分组。

响应长度预测。 以往响应长度预测工作主要关注非自回归生成 (NAR) 的翻译任务 [14]。这类任务一次生成整个句子, 因此预测响应长度十分重要。已有多种方法: [32] 利用数据集统计量和偏置项, [13] 预测每个输入 token 将被译成多少个 token; [9, 12] 等在编码器中加入特殊的 [LENGTH] token; [20, 21, 24, 31] 等用池化层和 MLP 分类器, 根据编码器输出预测响应长度。然而, 这些方法主要面向机器翻译, 其中目标序列长度与源序列长度接近, 更容易预测。本文则专门面向 LLM 推理, 不同查询类型及其响应长度变化很大。

3 响应长度感知

3.1 预先感知 (PiA)

经过指令微调的 LLM 能与人的理解对齐, 提供有帮助且安全的回答。我们发现, 这些模型似乎对即将生成的完整响应具有整体理解, 类似于人构思回答的过程。实验中, 我们要求模型预测即将生成的响应长度, 尽管预训练并未包含明确的响应长度感知任务。

如表 1 所示, 我们在原始提示中增加提前估计响应长度的指令, 将这一方法称为“预先感知” (Perception in Advance, PiA)。我们把修改后的提示用于多个 LLM, 观察其对指令的响应。

对响应长度的理解。 实验表明, 经过指令微调的 LLM 在收到 PiA 指令后, 具备较强的响应长度估计能力。我们使用 175 条 Alpaca 种子指令 [33] 测试 PiA。表 2 的结果表明, GPT-4、ChatGPT、Vicuna 对所有查询都能成功遵循指令。

一个有趣的发现是, LLM 对单词的理解优于对 token 的理解; 这与人理解和生成回答的方式一致。以单词为计量单位, 可以得到更可靠的响应长度估计。

为量化响应长度感知的准确性, 我们使用 Error(w), 衡量估计单词数与实际单词数之间的差异。还使用 Acc-50、Acc-100 两种阈值准确率: 误差小于 50 或 100 个单词时, 相应预测视为正确。GPT-4、Claude 的表现尤其突出, 平均误差小于 50 个单词, Acc-100 超过 90%。这些结果反映了模型生成的响应长度预测具有较高准确性和可靠性。此外, 参数更多的模型在响应长度预测上表现更好。

PiA 对响应生成的副作用。 引入 PiA 会影响响应生成过程。首先, 在生成阶段, 模型能够看到预计长度, 因此它可能影响模型如何生成回答。模型可能把预计长度视为约束, 尝试让回答适应该长度。这种行为可视为一种长度受限的生成。

表 2: 不同 LLM 采用预先感知方法的响应长度感知表现。Error(w) 为以单词计的误差; Acc-50、Acc-100 分别为误差小于 50、100 个单词的准确率。

	预先感知 (PiA)			仅感知 (PO)			失败率 ↓
	Error(w) ↓	Acc-50 ↑	Acc-100 ↑	Error(w) ↓	Acc-50 ↑	Acc-100 ↑	
GPT-4	22	80%	100%	100	28%	55%	0%
ChatGPT	51	77%	90%	89	55%	68%	2%
Claude	37	64%	96%	63	52%	84%	0%
Bard	70	44%	72%	130	28%	50%	28%
HugginChat-30B	77	52%	72%	113	56%	72%	12%
Vicuna-13B	94	49%	73%	92	55%	75%	0%
Vicuna-7B	123	40%	65%	122	40%	65%	0%

表 3: 基于 Vicuna 推理结果, 在 1 万条指令上的响应长度感知表现。

	误差 ↓	Acc-50 ↑	Acc-100 ↑
GPT-2 (1.5B)			
池化 + MLP	127	35%	53%
[LEN] token 微调	92	43%	64%
LLaMA-7B			
池化 + MLP	127	35%	53%
[LEN] token 微调	81	46%	70%
Vicuna-7B			
池化 + MLP	73	55%	75%
[LEN] token 微调	84	47%	72%
仅感知	193	38%	59%
指令微调	63	56%	81%

为研究 PiA 的影响, 我们比较 PiA 与“仅感知”(Perception Only, PO) 的响应长度感知误差。在 PO 中, 将未经修改的指令生成的响应长度, 与感知得到的长度值比较。注意, 同一指令在不同采样下会生成不同长度的回答。图 2b 展示了 1000 个数据点的长度变化, 说明可能长度的范围很广。因此, 响应长度并不存在唯一确定的“真实值”, 而是一个可能范围。为简化估计任务, 我们让模型预测潜在的最大长度; 使用平均长度存在局限, 后文将讨论。

对于 Vicuna 7B、13B 等较小 LLM, 我们观察到它们几乎忽略预计长度; GPT-4、Claude 则更倾向于让答案适应该长度, 因此误差明显较小。

此外, PiA 可能损害较小 LLM 的响应质量。例如, Vicuna-7B 有时预测完长度后就不再生成回答。这可能源于小模型同时处理多项任务的能力有限。

仅感知更困难。 为了避免预计长度影响响应生成的副作用, 我们在序列调度中采用“仅感知”(PO), 将预测与生成过程分离。

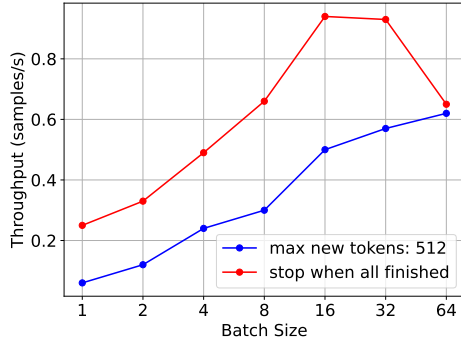
执行 PO 的直接方法是使用同一提示, 但只取长度预测; 再把估计长度与原始提示生成的实际回答比较。表 2 表明, LLM 仍能估计长度, 但与 PiA 相比, 误差明显更高, 准确率更低。

3.2 指令微调

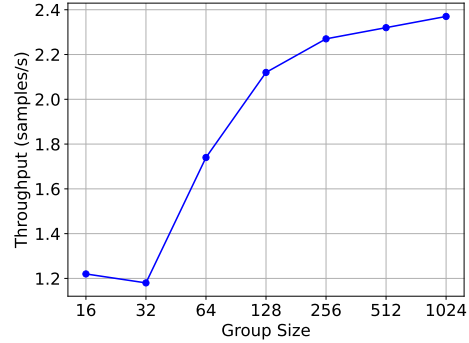
对于 GPT-4, PiA 的生成副作用可能较小, 已经足以支持序列调度; 但我们希望完全分离预测与生成, 避免预计长度的潜在影响。我们也希望较小模型能够准确感知响应长度。为实现这两个目标, 我们采用指令微调 [25]。

在指令微调阶段, 我们修改提示格式, 要求模型预测响应长度, 而非生成完整回答。从 Alpaca 数据集 [33] 选取 1 万条提示, 每条运行 4 次生成, 以观察到的最大长度作为目标。目标文本只有一个数字, 因此预测阶段的生成成本很小。为优化训练并节省计算资源, 我们采用高效的 LoRA [16] 方法; 与 LLM 相比, 其额外内存可以忽略。模型训练 3 个 epoch, 完整实验设置见附录。

表 3 展示指令微调带来的改进: 预测误差从 193 大幅降到 63, Acc-50 提高了 18%。我们也比较了 NAR 生成中已有的长度预测方法, 例如使用长度 token 微调, 以及用 MLP 对池化



(a) 固定生成长度时（蓝线），吞吐量近似线性增长；若等批次中所有响应完成才停止，大批次里的长响应会降低性能。



(b) 更大的查询组使调度更有效；查询组过小（32）时调度失效。随着查询组变大，继续加入指令的边际收益递减。

图 3: 吞吐量与批大小、查询组大小的关系。图内英文：Batch Size 为批大小，Group Size 为查询组大小，Throughput (samples/s) 为吞吐量（样本/秒）；max new tokens: 512 表示最多生成 512 个新 token，stop when all finished 表示全部完成后停止。

后的隐藏状态分类。虽然这些方法也有所改进，但不及指令微调。这说明模型能够理解并有效利用给定指令。此外，用 LLaMA-7B 或更小的 GPT-2 为 Vicuna 预测长度时，池化 + MLP 方法完全失效，长度 token 微调的表现也不及 Vicuna 的自预测能力。¹

4 序列调度

4.1 方法

建立较准确的响应长度感知模块后，可以利用它实施序列调度，提高推理效率。如图 1 左侧所示，将响应长度差异很大的指令放入同一批次，会产生大量冗余计算，降低吞吐量。因此，将响应长度相近的指令分组，可以加快推理。

在介绍具体调度方法前，先解释较大微批大小（mbs）的意义。图 3a 展示了在一张 80 GB A100 上部署 Vicuna 时，大批次如何更好地利用 GPU 并行计算能力。若每个样本固定生成相同数量的 token，吞吐量在批大小达到 16 之前近似线性增长，随后增速放缓。若每个批次等所有样本回答完成后才停止，在批大小小于 16 时，吞吐量也线性增长，而且增长比例高于固定 token 数的方式；但继续增大批次后，性能开始下降。原因是大批次更可能包含很长的回答，产生大量冗余计算。

为实现有效的序列调度，我们假设一次待处理的指令数量，即查询组大小，大于单张 GPU 的微批大小；在 LLM 广泛使用时这一假设通常成立。简单方法是将指令按预测长度排序，再切分为批次处理。我们进一步探索其他设计以提高吞吐量，流水线见 图 1 右侧。

失败收集与重新计算（FCR） 虽然长度预测器达到 81% 的 Acc-100，准确率较高，但有些样本的预测仍会明显偏离实际长度。这些错误可能严重破坏批处理效率。例如，长响应被误判为短响应、与大量短响应组批后，短查询就必须等待长查询，拖慢整个批次。为缓解这一问题，我们采用失败收集与重新计算（Failure Collection and Recomputation, FCR）：把新生成 token 数限制在批次内预测长度的最大值以内；超过预测长度仍未完成的指令视为失败，单独收集，等整个查询组推理结束后再重新计算。由于失败比例较低，这一方法能让短响应更快完成，同时只花较少时间重新生成失败指令。

可变批大小（VBS） 序列调度还需要考虑响应生成的内存使用。短响应比长响应所需内存少，但如果不考虑长响应被误判为短响应的可能而直接增大批次，就可能出现内存不足。借助失败重新收集，我们引入可变批大小（Variable Batch Size, VBS），为短响应分配更大的批次。一个简单规则是保持各批次 token 总数相同。设基准批大小为 B_0 ，对应长度为 L_0 ，

¹译注：Acc-50 从 38% 到 56% 是增加 18 个百分点。这里保留原文的“提高 18%”表述；表中 GPT-2 与 LLaMA 的池化 + MLP 结果也并非准确率为零，作者的“完全失效”是对其相对效果的描述。

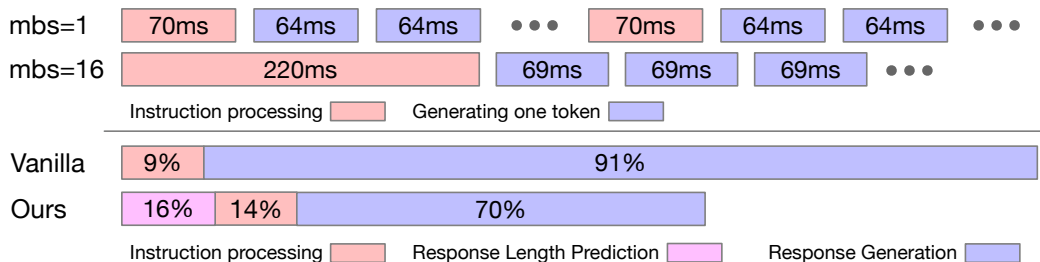


图 4: 上: 输入指令处理与 token 生成的时间比较。指令处理仅相当于生成少量 token 的时间。下: 原始方法与本文方法的各组件时间占比。图内英文: mbs 为微批大小, Instruction processing 为指令处理, Generating one token 为生成一个 token, Vanilla 为原始方法, Ours 为本文方法, Response Generation 为响应生成, Response Length Prediction 为响应长度预测; ms 为毫秒。

目标长度为 L , 原文给出的批大小调整公式为 $B_0 \cdot L / L_0$ 。该方法旨在让短响应采用更大的批大小, 优化内存使用, 同时防止 RC 引起内存溢出。²

分桶与预测最大长度 训练长度预测器时, 我们将目标长度分桶, 每个桶宽为 50, 将长度向上舍入到大于实际长度的最近桶。首先, 预测桶编号只需大致估计, 比预测精确数值更简单。其次, 分桶为长度预测误差提供缓冲; 只要实际长度仍落在同一桶内, 就不会影响方法效果。

此外, 我们预测 4 次生成中的最大长度, 因为低估的后果比高估更严重。通过预测最大长度, 我们希望预留足够容量来容纳尽可能长的输出。若预测平均长度, 可能低估部分查询, 使失败重新收集比例更高, 干扰组批过程。³

长度预测开销 我们必须在生成响应前, 预测查询组内所有指令的长度, 因此存在固有的预测开销。这包括计算指令 token 的键和值, 并生成少量 token, 通常为 2 至 3 个。然而, 如图 4 所示, 指令处理非常快, 只相当于生成少数 token 的时间, 约为 1 至 4 个。因此, 序列调度带来的总体加速可以抵消这部分开销。

4.2 实验

实验设置。 实验使用两个数据集: 从 Alpaca [33] 中选取 1 万条提示, 且与训练长度预测器所用子集不同; 以及 Instruction-in-Wild [36] 中的 429 条提示。前者为自指令方式生成的提示, 后者为从真实使用中收集的提示。

基线的批大小设为 16。对于可变批大小策略, 长度 (L) 不小于 300 的指令使用批大小 16; 小于 300 的指令按 $256 \times 50 / L$ 计算批大小, 再舍入到最近的 2 的幂, 以提高 GPU 利用率。查询组大小固定为 256, 在一张 80 GB A100 上使用 Vicuna-7B [4] 推理。生成温度设为 0.5, 以获得多样回答。

我们在样本层面评估吞吐量, 报告每秒处理的样本数, 以及相对于批大小为 16 的原始基线的提升。平均长度指各批次长度的均值; 平均长度更小意味着冗余计算更少。

结果。 表 4 比较不同响应长度感知模块的序列调度性能。在考虑的预测器中, 指令微调效果最好, 原文在这里报告吞吐量提高 85%。需要注意, 仅估计方法的失败重新收集比例明显更高, 降低了性能; 因此, 我们对该方法报告原始序列调度的结果。⁴

预测平均长度时, 失败重新收集比例升到 29.8%, 明显高于预测最大长度时的 15.3%, 性能提升因而降到 45%。如果把多次推理观察到的最大长度作为真实长度预测器, 则可视为本

²译注: 原文公式随目标长度增大而增大, 与“短响应采用更大批次”和保持 token 总数固定的描述相反。按该描述应反比调整; 下文实验确实使用与长度成反比的公式。此处保留原式。原文 RC 未在此明确展开, 应结合前述失败收集机制理解。

³译注: 4 次采样观察到的最大长度不是所有未来生成的严格上界。原文在此使用“最长可能输出”的说法, 实验只以有限次生成的最大值作为目标, 不能据此保证以后不会低估。

⁴译注: 此段的 85% 与表格、摘要及结论的 86% 不一致; 2.27 相对于 1.22 的提升约为 86.1%。原文两种表述均保留。

表 4: 采用不同响应长度感知模块的序列调度表现。

	吞吐量 (样本/秒) ↑	提升 ↑	平均长度 ↓
原始方法	1.22		377
真实长度预测器	2.52	+107%	201
池化 + MLP	1.96	+61%	216
[LEN] token 微调	2.10	+72%	210
仅感知 *	1.40	+15%	328
指令微调 (均值)	1.77	+45%	211
指令微调 (最大值)	2.27	+86%	208

表 5: 序列调度三个组件的消融实验。

分桶	FCR	VBS	吞吐量 (样本/秒) ↑	提升 ↑	平均长度 ↓
×	×	×	1.73	+42%	271
×	✓	×	1.76	+44%	222
×	✓	✓	2.22	+82%	209
✓	×	×	1.58	+30%	300
✓	✓	×	1.86	+52%	209
✓	✓	✓	2.27	+86%	203

文方法的一个性能上界。我们的方法仅比这一上界低 0.25 样本/秒，说明有效的长度预测器能带来较大收益。

此外，本文方法在三次实验中的吞吐量方差为 0.05，波动较低，体现了推理加速的稳定性和一致性。

表 5 展示三个组件的消融实验及其各自贡献。分桶增强了可变批大小 (VBS) 的效果，组合后更有效；每个组件都对最终吞吐量提升发挥了作用。表 6 进一步展示 Instruction-in-Wild 数据集的结果，验证本文方法在不同数据集上均有效。

此外，在图 3b 右侧，我们分析吞吐量与查询组大小的关系。查询组更大时，序列调度具有更大灵活性，吞吐量提高，但提升速度会逐渐放缓。因此，在真实场景中，需要平衡收集足够查询形成一个组的开销与吞吐量收益。

5 局限性与讨论

一个局限是，响应长度预测不一定准确，即使使用我们开发的、经过指令微调的长度预测模块也无法保证。实验结果具有潜力，但响应长度估计的准确性仍有提升空间。

此外，序列调度虽然显著减少了冗余计算，却不能完全消除这一问题。即使使用真实响应长度作为预测结果，冗余比例也只是从 66% 降到 33%，仍有很大的改进空间。ORCA [37] 等近期工作提出了新的推理系统，为缓解冗余计算问题提供其他方案。

另一个局限是，我们较少关注输入指令的处理。随着 LLM 支持的最大 token 长度增加，用户可能输入很长的指令，造成冗余计算与低效率。未来可探索把本文序列调度与输入组批调度技术 [10] 结合，进一步优化推理过程。

本文方法假设查询组大小超过单张 GPU 的容量。随着大型语言模型可能像搜索引擎一样成为普遍使用的基础设施，查询数量将大幅增长。此外，支持 32k 序列长度的 GPT-4、支持 100K 序列长度的 Claude 等模型，进一步加大了处理不同响应长度的挑战，凸显本文方法的相关性与有效性。

表 6: 在 Instruction-in-Wild 数据集上的序列调度表现。

	吞吐量 (样本/秒) ↑	平均长度 ↓	误差 ↓	Acc-50 ↑	Acc-100 ↑
原始方法	0.78	475	—	—	—
仅估计	1.07	422	358	20%	38%
指令微调	1.24	299	139	43%	68%

本文方法也容易扩展到多 GPU 场景，通过多张 GPU 加快处理并承载更大的模型。将感知响应长度不同的序列批次重新分配到不同 GPU 节点，可以使该方法继续适用于高性能计算环境。这种扩展有助于实现可扩展性，并保持本文获得的效率收益。

我们的发现表明，大型语言模型（LLM）对其生成的响应具有较全面的理解。这一能力为更快的推理技术创造了可能，包括能够克服逐 token 生成性能限制的非自回归方法。

6 结论

我们首先研究 LLM 能否在自回归解码前感知响应长度，并发现现有 LLM 在采用本文“预先感知”方法时表现良好。利用响应长度预测能力，我们提出序列调度来加速 LLM 推理，把响应长度相近的查询聚合成批次，减少计算浪费并提高吞吐量。我们进一步引入失败请求重新收集与可变批大小，以提高效率。在真实指令数据集上使用 Vicuna-7B 的实验表明，吞吐量提高了 86%，且没有牺牲输出质量。本文方法可补充现有 LLM 推理工具，帮助解决大规模高效部署的挑战。

致谢

Yang You 的研究组获得了 NUS 启动基金 (Presidential Young Professorship)、新加坡 MOE Tier-1 基金、ByteDance 基金、ARCTIC 基金、SMI 基金及 Alibaba 基金的资助。本工作由 Huawei Noah's Ark Lab 资助。

参考文献

- [1] R. Y. Aminabadi et al., “Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale,” *arXiv preprint arXiv:2207.00032*, 2022.
- [2] T. Brown et al., “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [3] Z. Cheng, J. Kasai, and T. Yu, “Batch prompting: Efficient inference with large language model apis,” *arXiv preprint arXiv:2301.08721*, 2023.
- [4] W.-L. Chiang et al., *Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality*, Mar. 2023. [Online]. Available: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [5] J. Choi, H. Li, B. Kim, S. Hwang, and J. H. Ahn, “Accelerating transformer networks through recomposing softmax layers,” in *2022 IEEE International Symposium on Workload Characterization (IISWC)*, IEEE, 2022, pp. 92–103.
- [6] A. Chowdhery et al., “Palm: Scaling language modeling with pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
- [7] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [10] J. Fang, Y. Yu, C. Zhao, and J. Zhou, “Turbotransformers: An efficient gpu serving system for transformer models,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 389–402.
- [11] E. Frantar, S. Ashkboos, T. Hoeffler, and D. Alistarh, “GPTQ: Accurate post-training compression for generative pretrained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [12] M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer, “Mask-predict: Parallel decoding of conditional masked language models,” *arXiv preprint arXiv:1904.09324*, 2019.
- [13] J. Gu, J. Bradbury, C. Xiong, V. O. Li, and R. Socher, “Non-autoregressive neural machine translation,” *arXiv preprint arXiv:1711.02281*, 2017.
- [14] J. Gu and X. Tan, “Non-autoregressive sequence generation,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, 2022, pp. 21–27.
- [15] J. Hoffmann et al., “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [16] E. J. Hu et al., “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [17] Y. Huang et al., “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” *Advances in neural information processing systems*, vol. 32, 2019.
- [18] S. Kim et al., “Full stack optimization of transformer inference: A survey,” *arXiv preprint arXiv:2302.14017*, 2023.
- [19] A. Koubaa, “Gpt-4 vs. gpt-3.5: A concise showdown,” 2023.
- [20] J. Lee, E. Mansimov, and K. Cho, “Deterministic non-autoregressive neural sequence modeling by iterative refinement,” *arXiv preprint arXiv:1802.06901*, 2018.
- [21] J. Lee, R. Shu, and K. Cho, “Iterative refinement in the continuous space for non-autoregressive neural machine translation,” *arXiv preprint arXiv:2009.07177*, 2020.

- [22] S. Li et al., “Colossal-ai: A unified deep learning system for large-scale parallel training,” *arXiv preprint arXiv:2110.14883*, 2021.
- [23] C. Lyu, J. Xu, and L. Wang, “New trends in machine translation using large language models: Case examples with chatgpt,” *arXiv preprint arXiv:2305.01181*, 2023.
- [24] X. Ma, C. Zhou, X. Li, G. Neubig, and E. Hovy, “Flowseq: Non-autoregressive conditional sequence generation with generative flow,” *arXiv preprint arXiv:1909.02480*, 2019.
- [25] L. Ouyang et al., “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [26] A. Paszke et al., “Automatic differentiation in pytorch,” 2017.
- [27] R. Pope et al., “Efficiently scaling transformer inference,” *arXiv preprint arXiv:2211.05102*, 2022.
- [28] J. Robinson, C. M. Rytting, and D. Wingate, “Leveraging large language models for multiple choice question answering,” *arXiv preprint arXiv:2210.12353*, 2022.
- [29] Y. Sheng et al., “High-throughput generative inference of large language models with a single gpu,” *arXiv preprint arXiv:2303.06865*, 2023.
- [30] M. Shoenberger, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [31] R. Shu, J. Lee, H. Nakayama, and K. Cho, “Latent-variable non-autoregressive neural machine translation with deterministic inference using a delta posterior,” in *Proceedings of the aaai conference on artificial intelligence*, vol. 34, 2020, pp. 8846–8853.
- [32] Z. Sun, Z. Li, H. Wang, D. He, Z. Lin, and Z. Deng, “Fast structured decoding for sequence models,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [33] R. Taori et al., *Stanford alpaca: An instruction-following llama model*, https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [34] H. Touvron et al., “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [35] X. Wu, C. Li, R. Y. Aminabadi, Z. Yao, and Y. He, “Understanding int4 quantization for transformer models: Latency speedup, composability, and failure cases,” *arXiv preprint arXiv:2301.12017*, 2023.
- [36] F. Xue, K. Jain, M. H. Shah, Z. Zheng, and Y. You, *Instruction in the wild: A user-based instruction dataset*, <https://github.com/XueFuzhao/InstructionWild>, 2023.
- [37] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {transformer-based} generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [38] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, “Benchmarking large language models for news summarization,” *arXiv preprint arXiv:2301.13848*, 2023.

附录

A 指令微调的其他实现细节

微调长度预测模块时，我们遵循以下流程。首先，每条指令采样 4 次来构建数据集：一次采用温度为 0 的贪心解码，另外三次采用温度 1.0，并使用不同随机种子。四次样本中的最大长度作为该指令的目标长度。

构建训练数据时，在原指令后追加只感知响应长度的要求。使用的提示为：“不要输出上面指令的回答。相反，你需要预测回答中的 token 数量。只输出一个数字。”

训练采用与 Vicuna [4] 对 LLaMA [34] 进行指令微调相同的超参数。具体而言，学习率为 0.00005，训练 3 个 epoch；LoRA [16] 仅应用于查询和键的线性层。训练在单张 80 GB A100 上进行，所有代码均使用 PyTorch [26] 实现。

B Instruction-in-Wild 数据集的分布

Instruction-in-Wild [36] 的响应长度直方图见图 5。该数据集的响应长度更多样，包含更多长响应。

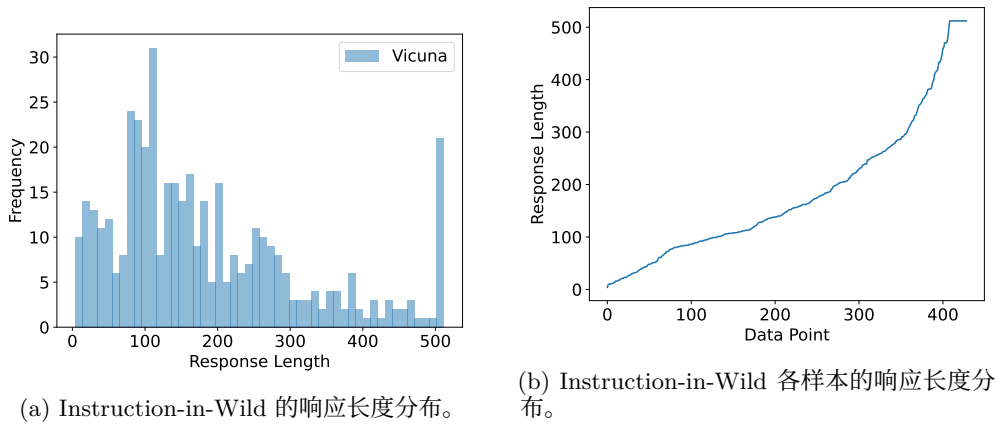


图 5: Instruction-in-Wild 的响应长度分布。图内英文：Response Length 为响应长度，Frequency 为频数，Data Point 为数据点。

C 采用 PiA 进行序列调度的讨论

正文介绍使用指令微调模型进行序列调度，以 LoRA 权重支持响应长度感知的方法。GPT-4、Claude 等近期 LLM 在预先感知 (PiA) 上也表现较好，因此可以直接利用 PiA 做序列调度，无需额外权重。

为进一步加快这种设置下的推理，一个潜在方法是在推理阶段复用响应长度感知阶段计算的输入查询键/值缓存 (KV 缓存)，避免重新计算输入指令的缓存，节省处理时间。

我们探索过一种策略：把 KV 缓存卸载到主存，再载入 GPU。然而，CPU 与 GPU 之间的传输时间可能很长，对总体性能影响很大，往往超过避免重算所节省的时间。一种可能的改进是只异步卸载与载入部分 KV 缓存，以减少传输开销；我们将其留待未来研究。

我们还尝试压缩 KV 缓存并直接保存在 GPU 内存中，采用 FlexGen 的量化方法 [29] 进行压缩，发现其对效果的影响较小。但这仍消耗额外内存，可能导致更小的批大小，降低总体性能。未来可以结合压缩与卸载，在内存使用和性能间取得平衡。

综合这些因素，本文流水线中的 PiA 响应长度感知模块继续采用重新计算策略。卸载与压缩仍有优化潜力，但需要进一步研究和改进，才能在实践中获得显著性能收益。

D 推理时间随 token 位置增加

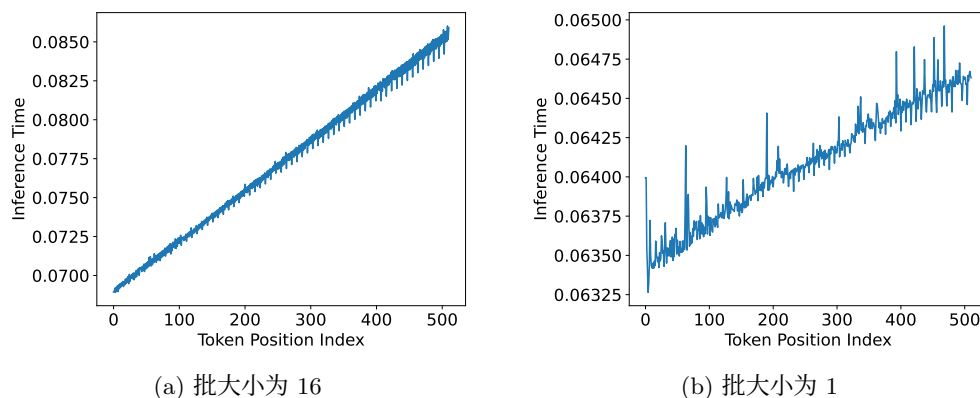


图 6: 推理时间随 token 位置索引增加。图内英文: Token Position Index 为 token 位置索引, Inference Time 为推理时间。

在 Transformer 中, 序列靠后的 token 需要对更多键和值执行自注意力操作, 因此推理时间通常更长。如图 6 所示, 所需推理时间随位置索引近似线性增加。因此, 在长序列中减少冗余计算, 比在短序列中节省相同数量的 token 更有效。这解释了为什么吞吐量的增长比例可能高于节省冗余 token 的比例。